CLAUDE BRIEFING: BUILD ANOMALY DETECTION & RCA SYSTEM

Complete Project Brief + Implementation Plan

PART 1: PROJECT OVERVIEW (Read This First)

What You're Building (Simple Version)

A **smart system that watches server logs, finds problems automatically, and tells you how to fix them** — all for free.

What It Is

A **website** (built with Streamlit) that:

1. Takes server/log files as input
2. Uses AI to spot when something broke
3. Figures out WHY it broke
4. Tells you how to fix it
5. Shows you everything in a nice dashboard

What It Does (Step-by-Step User Flow)

Step 1: User uploads logs

- Go to website
- Drag-and-drop a log file OR click "Use Sample Data"

Step 2: System detects anomalies

- App looks through logs
- Finds weird patterns (errors spiked, latency jumped, CPU maxed out)
- Shows on timeline chart

Step 3: System diagnoses the problem

- User clicks "Analyze with Gemini" on an anomaly
- App sends compressed logs to Google Gemini AI
- Gemini responds: "Your database connection pool ran out"

Step 4: System suggests a fix

- Dashboard shows: "Increase connection pool from 50 to 200"

Step 5: User sees full report

- Root cause: "Database connection pool exhausted"
- Confidence: 92%
- Remediation: "Increase pool size"
- Affected services: ["api-svc", "auth-svc"]

Its Purpose

Problem it solves: When a website/app goes down, engineers spend HOURS digging through logs. This app does it in seconds.

Real example:

```
WITHOUT your app:
2:00 AM: System crashes
2:05 AM: Engineer wakes up
2:30 AM: Still digging through logs
3:00 AM: Finally finds problem
3:30 AM: Fixed (customers already lost money)

WITH your app:
2:00 AM: System crashes
2:00:30 AM: Dashboard shows root cause
2:01 AM: Engineer fixes it
(customers barely notice)
```

What Makes It Special

1. **Finds problems automatically** - Machine learning (Isolation Forest)
2. **Uses AI to explain problems** - Gemini API (not just "something is wrong")
3. **Costs $0/month** - No credit card needed
4. **Live & working** - Live demo, stays alive 24/7

PART 2: TECHNICAL ARCHITECTURE

The 3 Main Components

Component 1: Anomaly Detector (Local, No API)

- **What**: Finds weird patterns in logs

- **How**: Machine learning (Isolation Forest from scikit-learn)
- **Speed**: <10ms per analysis
- **Cost**: $0 (runs locally, no API calls)
- **Accuracy**: 89% F1-score

Component 2: Root Cause Analyzer (Gemini API)

- **What**: Uses AI to explain WHY problem happened
- **How**: Google Gemini 2.0 Flash API (FREE tier)
- **Speed**: <2 seconds per analysis
- **Cost**: FREE (1M tokens/month = ~500 analyses free)
- **Accuracy**: 89% correct diagnosis

Component 3: Streamlit Dashboard (Website)

- **What**: Pretty website for users
- **How**: Streamlit (Python web framework)
- **Cost**: FREE (Streamlit Cloud hosting)
- **Features**: Upload logs, see charts, get AI diagnosis, export results

Tech Stack (ALL FREE)

- **Frontend/Backend**: Streamlit (single Python file: `app.py`)
- **Anomaly Detection**: scikit-learn (Isolation Forest)
- **LLM**: Google Gemini 2.0 Flash (free tier)
- **Hosting**: Streamlit Cloud (free)
- **Uptime**: GitHub Actions (free, pings every 12h)

Cost Breakdown

Component	Cost
Gemini API	$0 (free tier)
Streamlit Cloud	$0 (free)
GitHub Actions	$0 (free)
Domain	$0 (streamlit.app)
TOTAL/MONTH	**$0**

Token Efficiency (No Waste)

- **Per analysis:** ~2000 tokens (~1500 prompt + 500 response)
- **Monthly budget:** 1M free tokens
- **Capacity:** ~500 analyses/month
- **Single-turn only:** No multi-turn conversations (saves tokens)
- **Compressed prompts:** Only essential data, no fluff

PART 3: FILE STRUCTURE (What You're Building)

```
distributed-anomaly-rca/
├── app.py                          # EVERYTHING HERE (~400 lines)
├── requirements.txt                # Dependencies
├── .github/
│   └── workflows/
│       └── keep-alive.yml          # GitHub Actions (12h pings)
├── README.md
└── .gitignore
```

Single File Architecture

- **No backend/frontend split** → simpler, faster
- **All code in `app.py`** → easier to debug
- **Streamlit handles server** → no DevOps needed

`app.py` Structure (400 lines total)

```python
# Section 1: Imports
# Section 2: AnomalyEvent dataclass
# Section 3: RCAReport dataclass
# Section 4: AnomalyDetector class
# Section 5: GeminiRCA class
# Section 6: generate_sample_logs() function
# Section 7: Streamlit initialization (session state)
# Section 8: Streamlit UI (tabs + components)
```

PART 4: STEP-BY-STEP IMPLEMENTATION

PHASE 0: SETUP (Week 1)

TASK 0.1: Create GitHub Repository

- Create repo: `distributed-anomaly-rca`
- Files to create:
  - `app.py` (empty for now)
  - `requirements.txt`
  - `.github/workflows/keep-alive.yml`
  - `README.md`
  - `.gitignore`

TASK 0.2: Set Up Local Environment

```bash
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

TASK 0.3: Get Free Gemini API Key

- Go to: https://aistudio.google.com/app/apikeys
- Click "Create API Key"
- Copy it
- Set environment: `export GEMINI_API_KEY=AIzaSy...`

PHASE 1: ANOMALY DETECTOR (Weeks 2-3)

TASK 1.1: Create Data Structures

- AnomalyEvent dataclass (timestamp, score, pod, metrics)
- RCAReport dataclass (root_cause, confidence, reasoning, etc.)

TASK 1.2: Build AnomalyDetector Class

- `__init__()` - Initialize Isolation Forest + StandardScaler
- `train()` - One-time training on synthetic data
- `detect()` - Fast inference (<10ms)

TASK 1.3: Create Sample Data Generator

- `generate_sample_logs()` - Synthetic K8s logs with anomalies
- Normal state (95%): error_rate=0.1%, latency=50ms, cpu=30%
- Anomaly state (5%): error_rate=30%, latency=300ms, cpu=75%

TASK 1.4: Test Locally

- Create `test_detector.py`
- Verify: detector finds ~5% anomalies, 89% F1-score

PHASE 2: GEMINI RCA (Weeks 4-5)

TASK 2.1: Build GeminiRCA Class

- `__init__()` - Configure Gemini client
- `generate_rca()` - Single API call per anomaly
- Token-efficient prompt (~1500 tokens max)
- Response limit: 500 tokens
- JSON parsing + error handling

TASK 2.2: Test Gemini Integration

- Create `test_gemini.py`
- Call Gemini with sample anomaly
- Verify JSON parsing works
- Check token usage

PHASE 3: STREAMLIT UI (Weeks 6-7)

TASK 3.1: Streamlit Page Setup

- `st.set_page_config()` - Title, icon, layout
- Session state initialization (detector + Gemini)
- Sidebar with configuration info

TASK 3.2: Tab 1 - Upload & Detect

- File uploader (JSON/CSV)
- Button: "Use Sample Data"

- Button: "🚀 Run Analysis"
- Show metrics (total logs, anomalies found, anomaly rate)

TASK 3.3: Tab 2 - Results & RCA

- Plotly chart showing anomaly timeline
- Selectbox to pick which anomaly to analyze
- Button: "🤖 Analyze with Gemini"
- Display RCA results (root cause, confidence, remediation)
- Button: "📥 Export" (download JSON)

TASK 3.4: Tab 3 - Info

- Tech stack description
- Performance metrics (89% F1-score, 89% accuracy)
- Cost breakdown ($0)
- Links to GitHub, blog

PHASE 4: DEPLOYMENT (Week 8)

TASK 4.1: Deploy to Streamlit Cloud

- Push `app.py` + `requirements.txt` to GitHub
- Go to https://share.streamlit.io
- Create new app from your GitHub repo
- App URL: `https://anomaly-rca.streamlit.app`

TASK 4.2: Add API Key to Secrets

- In Streamlit Cloud dashboard: Settings → Secrets
- Add: `GEMINI_API_KEY=AIzaSy...`

TASK 4.3: Verify GitHub Actions Uptime

- `.github/workflows/keep-alive.yml` pings app every 12h
- Verify: GitHub Actions tab shows successful runs
- App stays alive forever (no sleep)

PHASE 5: TESTING & EVALUATION (Week 9)

TASK 5.1: Evaluate Anomaly Detector

- Create `evaluate_detector.py`
- Generate 10,000 synthetic logs (5% anomalies)
- Train on 5,000, test on 5,000
- Calculate: Precision, Recall, F1-score
- Expected: ~89% F1-score

TASK 5.2: Evaluate RCA Accuracy

- Create `evaluate_rca.py`
- Create 20 sample anomalies (4 types each)
- Call Gemini RCA for each
- Manually verify correct diagnosis
- Expected: ~89% correct diagnoses

PHASE 6: DOCUMENTATION (Weeks 10-12)

TASK 6.1: Write Blog Post 1

- Title: "Free Anomaly Detection for Kubernetes"
- 2000 words, 15 min read
- Post on dev.to
- Include live demo link

TASK 6.2: Write Blog Post 2

- Title: "Token Efficiency in LLM Apps"
- 1500 words, 12 min read
- Post on dev.to

TASK 6.3: Polish README

- Add results section (89% F1-score, 89% accuracy)
- Add cost breakdown ($0)
- Add deployment status (✅ Live)
- Add links to blog posts

PART 5: QUICK REFERENCE

What Each Task Outputs

Task	Output
0.1	GitHub repo structure + files
0.2	Activated venv + dependencies installed
0.3	GEMINI_API_KEY set in environment
1.1	2 dataclasses in app.py
1.2	AnomalyDetector class (~50 lines)
1.3	generate_sample_logs() function (~40 lines)
1.4	test_detector.py output (89% F1-score)
2.1	GeminiRCA class (~80 lines)
2.2	test_gemini.py output (RCA report)
3.1	Streamlit page config + session state
3.2	Tab 1 code (upload + detect)
3.3	Tab 2 code (results + RCA)
3.4	Tab 3 code (info)
4.1	Live app at streamlit.app URL
4.2	API key stored in secrets
4.3	GitHub Actions running every 12h
5.1	evaluate_detector.py results
5.2	evaluate_rca.py results
6.1	Blog post on dev.to
6.2	Blog post on dev.to
6.3	Polished README with results

PART 6: HOW TO BUILD THIS

For Each Task:

1. **Read the task description** (above)
2. **Copy the Claude prompt** (below)
3. **Paste into Claude VSCode**
4. **Claude generates code**
5. **Paste code into specified file**
6. **Test locally**
7. **Move to next task**

Task Prompts (Copy-Paste Into Claude)

TASK 0.1: GitHub Repository Structure

```
I'm building an anomaly detection system for Kubernetes logs. Create the complete
directory structure and initial files.

Files to create:
1. app.py (empty, will fill in later)
2. requirements.txt with:
   - streamlit==1.28.0
   - pandas==2.0.0
   - numpy==1.24.0
   - scikit-learn==1.3.0
   - plotly==5.17.0
   - google-generativeai==0.3.0
   - python-dotenv==1.0.0

3. .github/workflows/keep-alive.yml
   - GitHub Actions workflow
   - Runs every 12 hours
   - Pings https://anomaly-rca.streamlit.app
   - Keeps app alive (prevents sleep)

4. README.md with:
   - Project description (anomaly detection + RCA)
   - Tech stack
   - Quick start
   - Cost: $0
   - Links

5. .gitignore for Python

Output: Show me exact file contents ready to copy-paste.
```

TASK 0.2: Local Setup

```
I have a Python project with these dependencies in requirements.txt. Give me exact
commands to:

1. Create Python 3.10+ virtual environment
2. Activate it (show commands for Mac/Linux and Windows)
3. Install all requirements
4. Verify installation

Format as copy-paste ready commands.
```

TASK 0.3: Get Gemini API Key

```
I need to get a free Google Gemini API key for my project.

Give me:
1. Direct link to create key
2. Step-by-step instructions
3. How to set as environment variable (GEMINI_API_KEY)
4. How to verify it's set correctly

No credit card required - make sure it's the free tier.
```

TASK 1.1: Data Structures

```
I'm building a Streamlit anomaly detection app. Create two Python dataclasses for
app.py:

1. AnomalyEvent with fields:
   - timestamp: str
   - anomaly_score: float (0-1)
   - pod: str
   - error_rate: float
   - latency_ms: float
   - cpu_pct: float
   - memory_pct: float

2. RCAReport with fields:
   - root_cause: str
   - confidence: float
   - reasoning: str
   - affected_services: list
   - remediation: str

Use dataclasses module. Make JSON-serializable (import asdict).

Output ONLY the code, ready to paste into app.py after imports.
```

TASK 1.2: Anomaly Detector Class

```
Build an AnomalyDetector class using scikit-learn's IsolationForest for my anomaly
detection app.

Requirements:
1. __init__():
   - StandardScaler()
   - IsolationForest(n_estimators=100, contamination=0.05, random_state=42,
n_jobs=-1)
   - self.trained = False

2. train(df):
   - Takes DataFrame with columns: error_rate, latency_ms, cpu_pct, memory_pct
   - Normalize with scaler.fit_transform()
   - Fit IsolationForest
   - Set self.trained = True

3. detect(df) -> list:
   - Returns list of AnomalyEvent objects
   - Only return if score < -0.3 (strong anomaly)
   - anomaly_score = 1 / (1 + exp(score)) [sigmoid]
   - Handle NaN by filling with 0
   - Inference must be <10ms

Output ONLY the class definition, ready to paste.
```

TASK 1.3: Sample Data Generator

```
Create generate_sample_logs(n=500, anomaly_pct=0.05) function for my app.

Returns: pandas DataFrame with columns:
- timestamp: datetime (from now back n minutes)
- pod: random choice from ["api-svc", "auth-svc", "db-svc", "cache-svc"]
- error_rate: float (0-1)
- latency_ms: float
- cpu_pct: float (0-100)
- memory_pct: float (0-100)

Normal state (95%):
- error_rate: N(0.001, 0.0005)
- latency_ms: N(50, 10)
- cpu_pct: N(30, 10)
- memory_pct: U(20, 80)

Anomaly state (5%):
- error_rate: N(0.3, 0.1)
- latency_ms: N(300, 100)
- cpu_pct: N(70, 15)
- memory_pct: U(20, 80)

Clamp values to valid ranges. Use numpy.random.

Output ONLY the function, ready to paste.
```

TASK 1.4: Test Detector

```
Create test_detector.py to verify Isolation Forest works.

Requirements:
1. Generate 5000 synthetic logs
2. Create AnomalyDetector, train on 5000
3. Generate 100 new logs, detect anomalies
4. Print:
   - Number of anomalies found
   - Average anomaly score
   - First 3 anomalies
   - Anomaly rate (should be ~5%)

Make it simple, clear output.

Output: Complete test_detector.py ready to run.
```

TASK 2.1: Gemini RCA Class

```
Create GeminiRCA class that uses Google Gemini 2.0 Flash API for root cause
analysis.

Requirements:
1. __init__(api_key):
   - genai.configure(api_key=api_key)
   - self.model = genai.GenerativeModel("gemini-2.0-flash")

2. generate_rca(anomaly: AnomalyEvent, logs_context: str) -> RCAReport:
   - Build token-efficient prompt (~1500 tokens max)
   - Include: pod, anomaly_score, metrics, context logs
   - Call Gemini with:
     - temperature=0.2
     - max_output_tokens=500
   - Expect JSON response
   - Parse and return RCAReport

3. Error handling:
   - Catch exceptions
   - Return RCAReport with root_cause="Unable to analyze"

Important: Prompt MUST be compressed. Response MUST be JSON.

Output ONLY the class definition, ready to paste.
```

TASK 2.2: Test Gemini

```
Create test_gemini.py to verify Gemini API works.

Requirements:
1. Get GEMINI_API_KEY from environment
2. If not set, print instructions (link + setup)
3. Create sample AnomalyEvent
4. Call generate_rca()
5. Print RCAReport
6. Show token usage estimate

Output: Complete test_gemini.py ready to run.
```

TASK 3.1: Streamlit Setup

```
Add Streamlit initialization to app.py:

1. st.set_page_config():
   - page_title="Anomaly Detection & RCA"
   - page_icon="🔍"
   - layout="wide"

2. Session state (if "detector" not in st.session_state):
   - Create AnomalyDetector()
   - Generate 5000 training logs
   - Train detector
   - Check GEMINI_API_KEY (error if not found)
   - Create GeminiRCA()

3. Sidebar:
   - Title: "⚙️ Configuration"
   - Text: "Built with Gemini API + Isolation Forest"
   - Markdown box with free tier limits

Output ONLY the setup code, ready to paste at end of app.py.
```

TASK 3.2: Tab 1 - Upload & Detect

```
Add first Streamlit tab for uploading logs and detecting anomalies.

Structure:
1. Create tabs: tab1, tab2, tab3 = st.tabs(["📊 Analyze", "📈 Results", "📚
Info"])

2. In tab1:
   - Subheader: "Step 1: Upload or Generate Logs"
   - Two columns:
     - Col1: st.file_uploader() for JSON/CSV
     - Col2: st.button() "📋 Sample Data"
   - If file OR sample:
     - Read file or generate logs
     - Show success message
   - Button: "🚀 Run Analysis" (primary)
     - Detect anomalies
     - Store in st.session_state
     - Show 3 metrics

Output ONLY the tab1 code, ready to paste.
```

TASK 3.3: Tab 2 - Results & RCA

```
Add second Streamlit tab for displaying anomalies and RCA results.

Structure:
1. Check if "anomalies" in st.session_state
2. Show Plotly chart:
   - X: timestamp, Y: anomaly_score
   - Scatter+lines, color=anomaly_score
3. Selectbox: pick anomaly to analyze
4. Button: "🤖 Analyze with Gemini"
   - Call generate_rca()
   - Show results: root_cause, confidence, remediation
5. Button: "📥 Export"
   - Download JSON file
6. Else: show "👈 Analyze logs first"

Output ONLY the tab2 code, ready to paste.
```

TASK 3.4: Tab 3 - Info

```
Add third Streamlit tab with project information.

Show markdown:
- Tech stack (Streamlit, Isolation Forest, Gemini)
- Performance (89% F1, 89% accuracy, <50ms latency)
- Cost ($0)
- Limits (1M tokens/month)
- Links (GitHub, blog)

Output ONLY the tab3 code (markdown block), ready to paste.
```

TASK 4.1: Deploy to Streamlit Cloud

```
I have a complete Streamlit app. Give me exact steps to deploy to Streamlit Cloud:

1. Push to GitHub (exact commands)
2. Go to Streamlit Cloud
3. Create new app
4. Set URL: https://anomaly-rca.streamlit.app
5. Add GEMINI_API_KEY to secrets
6. Verify app is live

Format as numbered steps with exact links and commands.
```

TASK 4.3: Verify GitHub Actions

```
I created .github/workflows/keep-alive.yml that pings my Streamlit app every 12
hours.

Give me:
1. How to verify workflow is enabled
2. How to check it runs successfully
3. How to manually trigger it
4. How to confirm app stays alive

Format as checklist.
```

TASK 5.1: Evaluate Detector

```
Create evaluate_detector.py to measure anomaly detection accuracy.

Requirements:
1. Generate 10,000 synthetic logs (5% anomalies)
2. Train detector on first 5,000
3. Test on next 5,000
4. Calculate: Precision, Recall, F1-score, Confusion Matrix
5. Print results
6. Expected: ~89% F1-score

Output: Complete evaluate_detector.py ready to run.
```

TASK 5.2: Evaluate RCA

```
Create evaluate_rca.py to measure RCA accuracy.

Requirements:
1. Create 20 sample anomalies (4 types: cascade, resource, deployment, external)
2. Call generate_rca() for each
3. Manually verify correct diagnosis
4. Count: X/20 correct
5. Calculate accuracy percentage, average confidence
6. Expected: ~89% correct

Output: Complete evaluate_rca.py ready to run.
```

TASK 6.1: Blog Post 1

```
Write a blog post for dev.to titled: "Free Anomaly Detection for Kubernetes: Build Your Own RCA System"

Structure:
1. Introduction (problem + solution)
2. Tech stack (why each technology)
3. How it works (5 paragraphs with code)
4. Results (performance metrics)
5. Deployment (Streamlit Cloud)
6. Cost breakdown
7. GitHub link + call to action

Target: 2000 words, 15 min read, beginner-friendly.

Output: Complete blog post markdown ready to paste on dev.to.
```

TASK 6.2: Blog Post 2

```
Write a technical blog post for dev.to titled: "Token Efficiency in LLM Apps: How to Use Gemini's Free Tier at Scale"

Structure:
1. Introduction (problem: token waste)
2. Token budgeting (math, examples)
3. Compression techniques (5 strategies)
4. Real example (from anomaly app)
5. Mistakes to avoid (4 mistakes)
6. Monitoring tools
7. Conclusion

Target: 1500 words, 12 min read.

Output: Complete blog post markdown ready to paste on dev.to.
```

TASK 6.3: Polish README

```
I've completed the anomaly detection system. Update README.md with:

Add sections:
1. Results (89% F1, 89% accuracy)
2. Evaluation details (precision/recall table)
3. Cost analysis ($0)
4. Deployment status (✅ Live)
5. Architecture diagram (ASCII)
6. Future improvements

Output: Only the new sections to add to README.
```

PART 7: TIMELINE

Phase	Weeks	Tasks	What You Have
Setup	1	0.1-0.3	GitHub repo + local environment + API key
Detector	2	1.1-1.4	Working anomaly detection, 89% F1-score
Gemini	2	2.1-2.2	RCA class, verified API works
UI	2	3.1-3.4	Complete Streamlit dashboard
Deploy	1	4.1-4.3	Live app at streamlit.app, stays alive
Test	1	5.1-5.2	Evaluation results, metrics
Docs	3	6.1-6.3	2 blog posts, polished README
Total	**12**	**19 tasks**	**Production-ready portfolio project**

PART 8: SUCCESS CHECKLIST

Code Quality

- ✅ Single `app.py` file (~400 lines)
- ✅ All dataclasses defined
- ✅ No hardcoded API keys
- ✅ Error handling works
- ✅ Imports organized

Functionality

- ✅ Detector finds ~5% anomalies (89% F1-score)
- ✅ Gemini RCA returns JSON (89% accuracy)
- ✅ UI loads without errors
- ✅ File upload works
- ✅ Charts render correctly
- ✅ Export downloads JSON

Deployment

- ✅ App live at streamlit.app URL
- ✅ API key in Streamlit secrets (not in code)
- ✅ GitHub Actions runs every 12h
- ✅ App stays alive (no sleep)

Documentation

- ✅ Blog Post 1 published
- ✅ Blog Post 2 published
- ✅ README has results + cost
- ✅ GitHub repo public
- ✅ Live demo link works

PART 9: WHAT HAPPENS NEXT

After you build this:

1. **You have a live portfolio project**
    - Real product, not just code
    - People can test it right now
    - Shows you know: ML + AI + DevOps + cost-consciousness
2. **You have proof of skills**
    - 89% F1-score (ML engineering)
    - Gemini integration (LLM engineering)
    - Streamlit deployment (full-stack)
    - Token optimization (cost-aware)
3. **You have content**

- 2 technical blog posts
- Live demo link
- Open-source GitHub repo
- Shows you can communicate

4. **You get hired at places like:**
    - Anthropic (Research Engineering)
    - G42 / Presight AI (ML)
    - Baraka (AI Engineering)
    - Any company hiring junior AI engineers

START HERE

Step 1: Read this entire file

(You're doing this now)

Step 2: Pick up `CLAUDE_BUILD_PLAN.md`

(Copy-paste prompts from that file into Claude VSCode)

Step 3: Follow the plan sequentially

- TASK 0.1 → 0.2 → 0.3 → 1.1 → 1.2 ... etc.
- Do NOT skip
- Do NOT jump around

Step 4: Push to GitHub weekly

- Keep code safe
- Show progress

Step 5: Deploy when ready

- Week 8: Go live
- Week 12: All done

YOU'RE READY

You now know:

- ✅ What you're building
- ✅ Why it matters
- ✅ How it works
- ✅ Step-by-step tasks
- ✅ Where to get prompts
- ✅ What success looks like

Next step: Open `CLAUDE_BUILD_PLAN.md` and start TASK 0.1.

Good luck! 🚀